

Kubernetes, Docker & GitOps

Study Guide & Cheat Sheet

Concepts, commands, and worked examples — built from the `local-cicd-platform` project
(go-api · go-api-helm · Minikube · Harbor · Jenkins · ArgoCD · Prometheus/Grafana)

Docker

Kubernetes

Helm

Harbor

Jenkins

ArgoCD

Prometheus

Grafana

Trivy / gosec / gitleaks

Generated for internal learning & onboarding · every example is taken from this repository's actual files

Table of Contents

1. The Big Picture — how everything fits together
2. Docker — containers & image builds
3. Kubernetes — orchestration primitives
4. Helm — packaging & templating Kubernetes apps
5. Harbor — the container registry
6. Jenkins — continuous integration
7. ArgoCD — GitOps continuous delivery
8. Prometheus & Grafana — metrics & dashboards
9. Supporting security tooling (Trivy, gosec, govulncheck, gitleaks, golangci-lint)
10. Master Command Cheat Sheet
11. Troubleshooting Playbook
12. Glossary

1. The Big Picture

This project is a self-contained, single-laptop sandbox that reproduces a **real GitOps delivery pipeline**. It spans three separate git repositories, deliberately mirroring how production GitOps setups separate "app CI" from "cluster state":

Repo	Contains	Who touches it
<code>go-api</code>	Go/Gin REST API source, <code>Dockerfile</code> , <code>Jenkinsfile</code>	Developer pushes; Jenkins polls & builds
<code>go-api-helm</code>	Helm chart + ArgoCD <code>Application</code> / <code>ApplicationSet</code>	Jenkins pushes image-tag bumps; ArgoCD watches
<code>local-cicd-platform</code> (this repo)	Bootstrap scripts, root <code>Makefile</code> , infra installers	You, once, to stand the whole platform up

1.1 End-to-end flow (component view)



1.2 End-to-end flow (timeline / sequence view)

1. Developer runs `git push` against `go-api` on GitHub.
2. Jenkins' seed job `go-api-pipeline` polls the repo every ~5 minutes (`SCMTrigger('H/5 * * * *')`) and detects the new commit.
3. Pipeline runs `go mod download` → lint/format → static security scans → unit tests + benchmarks in parallel.
4. Pipeline builds a multi-stage Docker image tagged with the Jenkins `BUILD_NUMBER`.
5. Trivy scans the image; pipeline fails the build on any HIGH/CRITICAL CVE.
6. Pipeline logs into Harbor with the `harbor-robot` credential and pushes `:BUILD_NUMBER` and `:latest`.
7. Pipeline clones `go-api-helm` over SSH, runs `update-chart.sh` to bump `values.yaml`'s `image.tag` and `Chart.yaml`'s version, commits `[skip ci]`, and pushes to `main`.
8. ArgoCD reconciles `go-api-helm` on its default ~3-minute polling interval, diffs live cluster state against the new Git commit.
9. ArgoCD applies the delta (new image tag) — `automated: {prune: true, selfHeal: true}` means it self-corrects any manual drift too.
10. Kubernetes pulls the new image from Harbor (via `host.minikube.internal:8082` from inside the cluster), runs startup/liveness/readiness probes, and performs a zero-downtime rolling update.

11. The app's `/metrics` endpoint is scraped by Prometheus and visualized live in the pre-provisioned Grafana dashboard.

Why two different Git protocols? ArgoCD reads `go-api-helm` over plain **HTTPS** (public repo, read-only, no credentials needed). Jenkins writes to the same repo over **SSH** (needs push access). This asymmetry is a deliberate least-privilege split: the thing that only reads never holds write credentials.

1.3 Namespace layout

Namespace	Created by	Purpose
<code>go-api-dev</code>	ArgoCD (<code>CreateNamespace=true</code>)	1 replica, debug logs, ingress <code>go-api.dev.local</code> , ServiceMonitor on
<code>go-api-stage</code>	bootstrap.sh	2 replicas, ingress <code>go-api.stage.local</code>
<code>go-api-prod</code>	bootstrap.sh	3 replicas, HPA + PDB + NetworkPolicy on , TLS ingress
<code>argocd</code>	install-argocd.sh	ArgoCD control plane
<code>monitoring</code>	install-monitoring.sh	kube-prometheus-stack (Prometheus, Grafana, kube-state-metrics, node-exporter)
<code>ingress-nginx</code>	Minikube addon	Ingress controller

Bootstrap ordering matters: `scripts/bootstrap.sh` installs monitoring *before* ArgoCD, because the Helm chart's `ServiceMonitor` template is a Prometheus-Operator CRD — if ArgoCD tries to sync it before that CRD exists, the `Application` shows status `Unknown` .

2. Docker

2.1 Core concepts

Term	Meaning
Image	Immutable, layered filesystem + metadata used to spin up containers. Built from a <code>Dockerfile</code> .
Container	A running (or stopped) instance of an image — an isolated process with its own filesystem, network namespace, and cgroup limits.
Layer	Each Dockerfile instruction (<code>RUN</code> , <code>COPY</code> , ...) adds a cached, reusable filesystem layer.
Multi-stage build	Multiple <code>FROM</code> blocks in one Dockerfile; later stages copy only the artifacts they need from earlier ones, so build tools never ship in the final image.
Registry	A server that stores and serves images by <code>name:tag</code> (Docker Hub, Harbor, ECR, GHCR...).
Docker-outside-of-Docker (DooD)	Mounting the host's <code>/var/run/docker.sock</code> into a container so that container's <code>docker</code> CLI drives the <i>host's</i> daemon, instead of running a nested daemon (Docker-in-Docker).

2.2 In this project: `go-api/Dockerfile` (production, multi-stage)

```
# Stage 1: compile a static binary
FROM golang:1.21-alpine3.18 AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY cmd/ internal/ pkg/ ./
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \
    go build -ldflags="-w -s" -o /app/go-api ./cmd/api/main.go

# Stage 2: tiny runtime image, non-root user
FROM alpine:3.18.4
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=builder /app/go-api /app/go-api
USER appuser
EXPOSE 8080
HEALTHCHECK CMD wget --spider http://localhost:${PORT}/healthz || exit 1
ENTRYPOINT ["/app/go-api"]
```

Why this matters: stage 1's Go toolchain (~350 MB) never reaches the final image — only the compiled, stripped (`-ldflags="-w -s"`) static binary does. `CGO_ENABLED=0` means no libc dependency, so a minimal `alpine` base is enough. Running as a non-root `appuser` and dropping capabilities (enforced again later at the Kubernetes `securityContext` level) limits blast radius if the container is ever compromised.

2.3 Dev image: `Dockerfile.dev` + `docker-compose.yml`

A separate, single-stage image installs `air` (hot-reload) and mounts source code as a bind volume, so local edits recompile instantly without rebuilding the image — never used in the CI path, only for local development.

2.4 Command cheat sheet

Command	What it does
<code>docker build -t <repo>:<tag> .</code>	Build an image from the Dockerfile in the current directory
<code>docker images</code>	List local images
<code>docker ps</code> / <code>docker ps -a</code>	List running / all containers
<code>docker run -d -p 8080:8080 <image></code>	Run detached, mapping host:container port
<code>docker exec -it <container> sh</code>	Shell into a running container
<code>docker logs -f <container></code>	Stream container logs
<code>docker login <registry></code>	Authenticate to a registry (used with Harbor robot creds here)
<code>docker tag <src> <dst></code>	Add another tag to an existing image
<code>docker push</code> / <code>pull <repo>:<tag></code>	Upload / download an image to/from a registry
<code>docker compose up -d</code>	Start a multi-container stack defined in <code>docker-compose.yml</code>
<code>docker system prune -a</code>	Remove all unused images/containers/networks (frees disk)

3. Kubernetes

3.1 Core objects used in this project

Pod

Smallest deployable unit — one or more containers sharing network/storage. You rarely create Pods directly; a Deployment manages them.

Deployment

Declares desired replica count & Pod template; drives **RollingUpdate** to roll out new versions with zero downtime.

Service

Stable virtual IP + DNS name in front of a set of Pods, selected by label. This project uses `ClusterIP` (internal only).

Ingress

HTTP(S) routing rule mapping a hostname/path to a Service, handled by the ingress-nginx controller (a Minikube addon here).

ConfigMap

Non-secret key/value config injected as env vars or files. Holds `ENV`, `LOG_LEVEL`, `PORT` here.

Secret

Same idea as ConfigMap but for sensitive data (base64-encoded, not encrypted by default). Holds `AUTH_TOKEN` here.

Namespace

Logical partition of a cluster for multi-tenancy — this project uses one per environment (`go-api-dev/stage/prod`).

ServiceAccount / RBAC

Identity a Pod runs as, and `Role` / `RoleBinding` (namespace-scoped) or `ClusterRole` / `ClusterRoleBinding` (cluster-scoped) grants for that identity.

HPA (HorizontalPodAutoscaler)

Automatically scales replica count based on observed metrics (CPU % here).

PDB (PodDisruptionBudget)

Guarantees a minimum number of Pods stay up during voluntary disruptions (node drains, cluster upgrades).

NetworkPolicy

Firewall rules for Pod-to-Pod / Pod-to-external traffic, enforced by the CNI.

PVC (PersistentVolumeClaim)

A request for durable storage, bound to a PersistentVolume by a StorageClass.

3.2 Probes — how Kubernetes knows a Pod is healthy

Probe	Purpose	In this chart
startupProbe	Gives slow-starting apps grace time before liveness kicks in	<code>GET /livez</code> , every 2s, 10 tries
livenessProbe	Restarts the container if it stops responding	<code>GET /livez</code> , every 10s
readinessProbe	Removes the Pod from Service endpoints if not ready to serve traffic (without restarting it)	<code>GET /readyz</code> , every 5s

3.3 Deployment manifest, annotated (go-api-helm/templates/deployment.yaml)

```
spec:
  strategy:
    type: RollingUpdate      # replace pods gradually, never all-at-once
    rollingUpdate:
      maxSurge: 25%         # up to 25% extra pods during rollout
      maxUnavailable: 25%   # at most 25% of pods down at once
  template:
    spec:
      securityContext:       # pod-level
        fsGroup: 2000
      containers:
        - securityContext:   # container-level, more restrictive
            capabilities: { drop: [ALL] }
            runAsNonRoot: true
            runAsUser: 10001
            image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
            env:
              - name: AUTH_TOKEN
                valueFrom: { secretKeyRef: { name: go-api-secret, key: AUTH_TOKEN } }
            startupProbe: { httpGet: { path: /livez, port: http } }
            livenessProbe: { httpGet: { path: /livez, port: http } }
            readinessProbe: { httpGet: { path: /readyz, port: http } }
```

3.4 Networking model in this project

The chart's Service is `ClusterIP` (port 80 → container port 8080). Ingress (nginx class) is only **enabled by default in dev**; staging/prod also enable it with environment-specific hostnames (`go-api.stage.local` , `go-api.prod.local` , TLS redirect on in prod). Locally, `make port-forward` exposes each environment's Service directly on the host (dev 8081, stage 8083, prod 8084) — bypassing Ingress entirely for quick access.

3.5 Command cheat sheet

Command	What it does
<code>kubectl get pods -A</code>	List Pods across all namespaces

<code>kubectl get pods -n go-api-dev -o wide</code>	List Pods in one namespace with node/IP columns
<code>kubectl describe pod <name> -n <ns></code>	Full details + recent Events (first stop for debugging)
<code>kubectl logs -f <pod> -n <ns></code>	Stream a Pod's logs
<code>kubectl logs -l app.kubernetes.io/name=go-api -n go-api-dev -f</code>	Stream logs from all Pods matching a label (what <code>make logs-dev</code> runs)
<code>kubectl exec -it <pod> -n <ns> -- sh</code>	Shell into a running Pod
<code>kubectl apply -f <file></code>	Create/update resources declaratively from a manifest
<code>kubectl delete -f <file></code>	Remove resources defined in a manifest
<code>kubectl get events -n <ns> --sort-by=.lastTimestamp</code>	See what the control plane has been doing recently
<code>kubectl port-forward svc/<svc> -n <ns> 8081:80</code>	Tunnel a local port to a cluster Service (no Ingress needed)
<code>kubectl rollout status deployment/<name> -n <ns></code>	Watch a rolling update until it completes
<code>kubectl rollout undo deployment/<name> -n <ns></code>	Roll back to the previous ReplicaSet
<code>kubectl get hpa -n <ns></code>	Check autoscaler target vs. current CPU utilization
<code>kubectl top pods -n <ns></code>	Live CPU/mem usage (needs metrics-server, installed here)
<code>kubectl get ns</code>	List namespaces
<code>minikube start --driver=docker --insecure-registry="host.minikube.internal:8082" --addons=ingress,dashboard</code>	Bring up the local cluster used by this project, allowing HTTP-only Harbor pulls
<code>minikube dashboard</code>	Open the web UI for the cluster

4. Helm

4.1 Core concepts

Term	Meaning
Chart	A packaged bundle of Kubernetes templates + metadata (<code>Chart.yaml</code>) + default config (<code>values.yaml</code>). This project has one chart: <code>go-api-helm</code> .
Values file	YAML overrides fed into the chart's templates. Helm merges multiple <code>-f</code> files left-to-right, later files win.
Template	A Kubernetes manifest with Go-template placeholders (<code>{{ .Values.x }}</code>), rendered at install/upgrade time.
Release	A named, versioned instance of a chart installed into a cluster/namespace (e.g. <code>go-api-dev</code>).
Helper template (<code>_helpers.tpl</code>)	Reusable named templates (e.g. <code>go-api.fullname</code> , <code>go-api.labels</code>) shared across manifests to avoid repetition.

4.2 Environment overlay pattern used here

Rather than one values file per cluster, this chart layers a shared base over an environment-specific overlay — exactly what ArgoCD's `helm.valueFiles` list does:

```
helm.valueFiles:
- values.yaml           # shared defaults
- values-dev.yaml       # environment-specific overrides, applied last (wins)
```

File	Replicas	Log level	Notable overrides
<code>values.yaml</code> (base)	1	info	image repo/tag, security context, all feature flags off by default
<code>values-dev.yaml</code>	1	debug	Ingress on (<code>go-api.dev.local</code>), ServiceMonitor on (only env with metrics scraping by default)
<code>values-stage.yaml</code>	2	info	Mid-tier resources, ingress host <code>go-api.stage.local</code>
<code>values-prod.yaml</code>	3	warn	Autoscaling (3–10 @75% CPU), PDB (<code>minAvailable: 2</code>), NetworkPolicy on, TLS ingress

Learning note: `values-prod.yaml` hardcodes a mock `authToken` with a comment admitting: "In enterprise setups, this is injected via vault/external secrets." This repo intentionally does **not** implement Vault/External-Secrets — a good next exercise once you're comfortable with the basics.

4.3 Key templates in `go-api-helm/templates/`

Template	Renders	Gated by
<code>deployment.yaml</code>	Deployment with probes, env-from-ConfigMap/Secret, security context	always
<code>service.yaml</code>	ClusterIP Service	always
<code>configmap.yaml</code> / <code>secret.yaml</code>	App config & auth token	always
<code>ingress.yaml</code>	Ingress rule(s)	<code>ingress.enabled</code>
<code>hpa.yaml</code>	HorizontalPodAutoscaler (autoscaling/v2)	<code>autoscaling.enabled</code>
<code>pdb.yaml</code>	PodDisruptionBudget	<code>pdb.enabled</code>
<code>networkpolicy.yaml</code>	Ingress/egress firewall rules	<code>networkPolicy.enabled</code>
<code>pvc.yaml</code>	PersistentVolumeClaim	<code>pvc.enabled</code>
<code>rbac.yaml</code>	Role/RoleBinding + ClusterRole/ClusterRoleBinding	always (bound to the app's own ServiceAccount)
<code>servicemonitor.yaml</code>	Prometheus Operator scrape config for <code>/metrics</code>	<code>serviceMonitor.enabled</code>
<code>grafana-dashboard.yaml</code>	ConfigMap holding a pre-built dashboard JSON, auto-loaded by Grafana's sidecar	<code>serviceMonitor.enabled</code> (shared flag)
<code>NOTES.txt</code>	Post-install "how to access your app" printout	—

4.4 Command cheat sheet

Command	What it does
<code>helm template . -f values.yaml -f values-dev.yaml</code>	Render manifests locally without installing — best way to sanity-check a chart change
<code>helm upgrade --install go-api-dev . -f values.yaml -f values-dev.yaml -n go-api-dev</code>	Install if new, upgrade if it exists (idempotent) — used to deploy this chart manually
<code>helm lint .</code>	Static-check a chart for common mistakes
<code>helm list -n <ns></code>	Show releases installed in a namespace
<code>helm status <release> -n <ns></code>	Show a release's current state
<code>helm rollback <release> <revision> -n <ns></code>	Roll back to a previous release revision
<code>helm uninstall <release> -n <ns></code>	Remove a release and its resources
<code>helm repo add prometheus-community https://prometheus-community.github.io/helm-charts</code>	Register a chart repository (used to install kube-prometheus-stack)

```
helm get values <release> -n <ns>
```

Show the effective values currently in use by a release

5. Harbor

5.1 Core concepts

Term	Meaning
Registry	Harbor itself — an OCI-compliant image registry with a web UI, RBAC, and vulnerability scanning built in (an open-source alternative to Docker Hub / ECR / GCR).
Project	A namespace inside Harbor grouping repositories, with its own access policy (public/private) and quotas. This project uses one project: <code>library</code> .
Robot account	A non-human, scoped credential (e.g. <code>robot\$jenkins-robot</code>) used by CI systems instead of a real user's password.
Trivy adapter	An optional Harbor component giving the registry its own built-in CVE scanner UI, independent of any scanner run by CI.

5.2 How this project's Harbor is configured

Installed by `scripts/install-harbor.sh` (Harbor v2.8.2, official online installer), then patched via `sed` into:

```
hostname: localhost
http:
  port: 8082          # moved off the default 80 to avoid clashing with Jenkins on 8080
harbor_admin_password: Harbor12345
# https: block fully commented out -> HTTP-only registry
```

Why HTTP-only matters: container runtimes refuse to pull from a plain-HTTP registry by default ("insecure registry"). That's why Minikube in this project is started with `--insecure-registry="host.minikube.internal:8082"` — explicitly whitelisting Harbor as an exception. In production you would terminate TLS instead.

5.3 Dual addressing — one registry, two hostnames

Caller	Address used	Why
Jenkins (host Docker daemon)	<code>localhost:8082</code>	Jenkins' Docker CLI talks to the host daemon directly (Docker-outside-of-Docker)
Kubelet inside Minikube	<code>host.minikube.internal:8082</code>	From inside the Minikube VM, <code>localhost</code> means the VM itself, not your laptop — Minikube provides this DNS alias to reach the host

5.4 The robot account (`scripts/create-harbor-project.sh`)

Automated purely via Harbor's REST API (`http://localhost:8082/api/v2.0`): creates the public `library` project, then a system-level robot `jenkins-robot` scoped to `repository:push` , `repository:pull` , `artifact:create` , `artifact:delete` on that project only — never full-admin access. Credentials land in `infrastructure/harbor-robot.env` (gitignored, chmod 600):

```
HARBOR_REGISTRY=localhost:8082
HARBOR_PROJECT=library
HARBOR_ROBOT_USER=robot$jenkins-robot
HARBOR_ROBOT_SECRET=<generated>
```

Gotcha: the robot username literally contains a `$` character. Anything that later `source` s this env file must quote the variable — unquoted shell expansion would try to interpret `$jenkins-robot` as a variable reference.

5.5 Command cheat sheet

Command	What it does
<code>docker login localhost:8082</code>	Authenticate to Harbor (robot or admin creds)
<code>docker push localhost:8082/library/go-api:42</code>	Push an image to a Harbor project/repo
<code>curl http://localhost:8082/api/v2.0/ping</code>	Health-check the Harbor API (used by bootstrap polling)
<code>curl -u admin:Harbor12345 http://localhost:8082/api/v2.0/projects</code>	List Harbor projects via REST API
Harbor UI → Project → Repositories → tag → Vulnerability tab	View Trivy-adapter scan results in-browser

6. Jenkins

6.1 Core concepts

Term	Meaning
Declarative Pipeline	A <code>Jenkinsfile</code> written in a structured <code>pipeline { stages { ... } }</code> DSL, checked into the app repo (pipeline-as-code).
Stage	A named phase of the pipeline (Lint, Test, Build, Scan, Push...), shown as a step in Jenkins' UI ("Blue Ocean" style graph).
Agent	Where a stage/pipeline actually executes. This project uses <code>agent any</code> — run directly on the single Jenkins controller container (no separate build agents).
Credentials store	Jenkins' encrypted vault for secrets (SSH keys, username/password pairs), referenced by ID in the <code>Jenkinsfile</code> — never hardcoded.
SCM polling trigger	Jenkins periodically checks the source repo for new commits (vs. a webhook, which pushes notifications instantly). This project polls every ~5 min via cron syntax <code>H/5 * * * * .</code>
init.groovy.d	Groovy scripts a Jenkins controller auto-runs on JVM startup, before the setup wizard — used here for fully headless (no-click) provisioning.

6.2 Headless provisioning (`scripts/install-jenkins.sh`)

Rather than clicking through Jenkins' setup wizard, three Groovy scripts are dropped into `jenkins_home/init.groovy.d/` and run automatically on boot:

Script	Does
<code>00-security.groovy</code>	Creates the admin user, enables a private security realm, disables anonymous read access
<code>10-credentials.groovy</code>	Registers <code>github-personal-ssh</code> (SSH key) and <code>harbor-robot</code> (username/password) into the global credentials store
<code>20-seed-job.groovy</code>	Programmatically creates the <code>go-api-pipeline</code> job pointed at your fork, with the SCM poll trigger — and explicitly starts that trigger

Real bug fixed in this repo's git history: the seed job originally built its `SCMTrigger` object but never called `trigger.start(job, true)` before `job.save()` — so the trigger existed in config but never actually scheduled polls. The fix (commit "*Fix seed pipeline job's SCM poll trigger not activating*") added that missing call. A good reminder that configuring an object isn't the same as activating it.

The Jenkins image itself (`infrastructure/jenkins/Dockerfile`) is `jenkins/jenkins:lts-jdk17` plus the Docker CLI (bind-mounted host socket = Docker-outside-of-Docker), Go, and the security toolchain (`gosec` , `trivy` , `golangci-lint` , `govulncheck` , `gitleaks`) baked in so no stage needs to install tools on the fly.

6.3 The pipeline, stage by stage (go-api/Jenkinsfile)

```
pipeline {
  agent any
  environment {
    REGISTRY = "localhost:8082"
    IMAGE_NAME = "library/go-api"
  }
  stages {
    stage('Lint & Format')           // gofmt, go vet, golangci-lint
    stage('Static Security Analysis') // gosec, govulncheck, gitleaks
    stage('Testing') { parallel { // unit tests + benchmarks concurrently
      stage('Unit Tests') { sh "go test -v -race -coverprofile=coverage.out ./..." }
      stage('Benchmarks') { sh "go test -bench=. -run=^$ ./..." }
    } }
    stage('Docker Build') // docker build over the DooD socket
    stage('Trivy Image Scan') // exit-code 1 on HIGH/CRITICAL CVE = fails build
    stage('Push to Harbor') // docker login + push :BUILD_NUMBER and :latest
    stage('Update GitOps Chart') { sshagent(['github-personal-ssh']) {
      // clone go-api-helm, bump image.tag via update-chart.sh, commit, push
    } }
  }
}
```

Quality gates that actually fail the build: unformatted code (`gofmt -l .` non-empty), `go vet` / `golangci-lint` issues, and any HIGH/CRITICAL Trivy finding. **Non-blocking:** `gitleaks detect` failures are caught with `|| echo ...` — intentionally advisory only in this sandbox.

6.4 GitOps handoff — go-api/scripts/update-chart.sh

```
# Usage: update-chart.sh <chart-dir> <image-tag> <app-version> [chart-version]
yq eval -i '.image.tag = "'"$IMAGE_TAG"'" values.yaml
yq eval -i '.appVersion = "'"$APP_VERSION"'" Chart.yaml
# if no chart version given: auto-bump Chart.yaml's patch version (x.y.Z -> x.y.Z+1)
```

6.5 Command / concept cheat sheet

Item	What it does
<code>Build Now</code> (UI) or <code>curl -X POST .../job/<job>/build</code>	Trigger a pipeline run manually, without waiting for the poll
<code>H/5 * * * * *</code>	Cron-style SCM poll every 5 minutes (<code>H</code> = hashed offset to spread load)
<code>withCredentials([usernamePassword(...)])</code>	Injects a stored credential as env vars for the duration of a step block

<code>sshagent(['cred-id']) { ... }</code>	Loads an SSH private key into an agent for git/ssh operations inside the block
<code>docker exec -it jenkins-local bash</code>	Shell into the running Jenkins container to debug
<code>docker logs -f jenkins-local</code>	Tail Jenkins controller logs

7. ArgoCD

7.1 Core concepts — what "GitOps" actually means

GitOps: Git is the single source of truth for desired cluster state. Instead of a CI job running `kubectl apply` imperatively, a controller (ArgoCD) continuously *pulls* the desired state from Git and reconciles the live cluster to match it — including reverting manual `kubectl` changes ("self-heal") if drift is detected.

Term	Meaning
Application (CRD)	ArgoCD's core object: binds a Git source (repo/path/revision) to a cluster destination (server/namespace).
Sync	The act of reconciling live state to match Git. Can be manual (click "Sync") or <code>automated</code> .
Prune	When automated, delete cluster resources that were removed from Git.
Self-heal	When automated, revert any manual/out-of-band change back to what Git declares.
ApplicationSet	A generator that stamps out multiple <code>Application</code> s from one template (e.g. one per environment).
App-of-Apps	A common pattern (not used in this project) where one root Application manages a set of child Applications.

7.2 This project's `Application` (go-api-helm/argocd/application.yaml)

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: go-api-dev
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/SakthiSP333/go-api-helm.git # HTTPS, read-only
    targetRevision: main
    helm:
      valueFiles: [values.yaml, values-dev.yaml]
  destination:
    server: https://kubernetes.default.svc
    namespace: go-api-dev
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [CreateNamespace=true]
```

Only apply one of the two ArgoCD manifests. `application.yaml` (single dev Application, applied by `install-argocd.sh`) and `application-set.yaml` (fans out dev/stage/prod via a `list` generator) both try to own an Application named `go-api-dev` — applying both causes a conflict. Pick one strategy per cluster.

7.3 Install mechanism used here

Unlike the monitoring stack, ArgoCD is installed from the **official upstream manifest** rather than a Helm chart:

```
kubectl create namespace argocd
kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Admin credentials come from the auto-generated `argocd-initial-admin-secret` Kubernetes Secret — retrieved once at install time and saved to `infrastructure/argocd-creds.env`.

7.4 Command cheat sheet

Command	What it does
<code>kubectl port-forward svc/argocd-server -n argocd 8085:443</code>	Reach the ArgoCD UI/API locally (what <code>make argocd-ui</code> runs)
<code>kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath='{.data.password}' \ base64 -d</code>	Retrieve the initial admin password
<code>argocd login localhost:8085</code>	Authenticate the ArgoCD CLI
<code>argocd app list</code>	List all registered Applications
<code>argocd app get go-api-dev</code>	Show sync/health status & resource tree for one Application
<code>argocd app sync go-api-dev</code>	Force an immediate sync instead of waiting for the poll
<code>argocd app diff go-api-dev</code>	Show what would change between Git and the live cluster
<code>argocd app history go-api-dev</code>	List past sync revisions (for rollback)
<code>argocd app rollback go-api-dev <id></code>	Revert to a previous synced revision
UI → Application → Refresh	Force ArgoCD to re-fetch Git and recompute diff (used after fixing the ServiceMonitor CRD ordering issue)

8. Prometheus & Grafana

8.1 Core concepts

Term	Meaning
Prometheus	Pull-based metrics database: it scrapes HTTP <code>/metrics</code> endpoints on a schedule and stores time-series data.
Counter	A metric that only goes up (e.g. total requests). Use <code>rate()</code> in queries to get a per-second rate.
Histogram	Buckets observations (e.g. request duration) so you can compute quantiles like p95/p99 after the fact.
ServiceMonitor (CRD)	Prometheus-Operator's declarative "scrape this Service" config — the Kubernetes-native way to tell Prometheus what to scrape, instead of hand-editing <code>prometheus.yml</code> .
PromQL	Prometheus' query language, used both for alerting rules and Grafana panels.
Grafana	Visualization layer on top of Prometheus (and other data sources) — dashboards, panels, alerting.

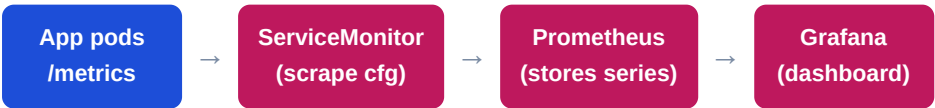
8.2 Metrics instrumentation in the app (go-api/internal/middleware/metrics.go)

```
http_requests_total           // CounterVec, labels: method, path, status
http_request_duration_seconds // HistogramVec, default buckets

// wired into Gin as middleware, exposed unauthenticated at GET /metrics:
router.GET("/metrics", gin.WrapH(promhttp.Handler()))
```

Note `/metrics` sits *outside* the `/api/v1` auth group — Prometheus scrapes it with no bearer token, matching the convention that metrics endpoints are typically network-isolated rather than app-authenticated.

8.3 Wiring it up: ServiceMonitor → Prometheus → Grafana



The chart's `serviceMonitor.enabled` flag (true only in `values-dev.yaml` by default) gates *both* the `ServiceMonitor` template and the `grafana-dashboard.yaml` ConfigMap — they're only meaningful together, once the monitoring stack exists in-cluster.

8.4 The pre-built Grafana dashboard panels

Panel	PromQL
-------	--------

Request rate by status	<code>sum by (status) (rate(http_requests_total{...}[5m]))</code>
p95 latency	<code>histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))</code>
5xx error rate	Same request-rate query filtered to <code>status=~"5.."</code>

Auto-discovery works because `install-monitoring.sh` enables Grafana's sidecar (`grafana.sidecar.dashboards.enabled=true` , `searchNamespace=ALL`) which watches for any ConfigMap labeled `grafana_dashboard: "1"` cluster-wide and loads its JSON automatically — no manual dashboard import needed.

8.5 Laptop-friendly resource tuning

`install-monitoring.sh` installs the `kube-prometheus-stack` Helm chart with persistence disabled, a short `6h` retention window, tiny CPU/memory requests across every component, and Alertmanager disabled entirely — this is a learning sandbox, not a durable observability stack.

8.6 Command cheat sheet

Command	What it does
<code>helm upgrade --install kube-prometheus-stack prometheus-community/kube-prometheus-stack -n monitoring --create-namespace</code>	Installs Prometheus + Grafana + Alertmanager + exporters as one chart
<code>kubectl port-forward svc/kube-prometheus-stack-prometheus -n monitoring 9090:9090</code>	Reach the Prometheus UI (what <code>make prometheus-ui</code> runs)
<code>kubectl port-forward svc/kube-prometheus-stack-grafana -n monitoring 3000:80</code>	Reach the Grafana UI (what <code>make grafana-ui</code> runs)
Prometheus UI → Status → Targets	Verify your ServiceMonitor is actually being scraped (green = up)
Prometheus UI → Graph , type a PromQL query	Ad-hoc metric exploration before building a Grafana panel
<code>kubectl get servicemonitor -A</code>	List all registered scrape configs cluster-wide
<code>curl http://localhost:8081/metrics</code>	See the raw metrics text format the app exposes

9. Supporting Security & Quality Tooling

Tool	Category	What it checks	Used where
golangci-lint	Linters	Aggregates dozens of Go linters (style, bugs, unused code) into one run	Jenkinsfile "Lint & Format" stage
gosec	SAST	Scans Go source for common security anti-patterns (SQL injection, weak crypto, etc.)	Jenkinsfile "Static Security Analysis"
govulncheck	SCA	Cross-references your dependency graph against the official Go vulnerability database	Same stage
gitleaks	Secret scanning	Greps git history/diffs for accidentally-committed credentials/keys	Same stage (non-blocking here)
Trivy	Container/image scanner	CVEs in OS packages & language dependencies inside a built image	Jenkinsfile "Trivy Image Scan" (blocking, HIGH/CRITICAL) <i>and</i> Harbor's built-in scanner UI
syft	SBOM generator	Produces a Software Bill of Materials for an image	Baked into the Jenkins image but not currently invoked by any stage — a good exercise to wire in

Defense in depth, twice over: vulnerability scanning happens in two independent places — once in CI (Trivy CLI, blocking) and again inside Harbor itself (Trivy adapter, browsable per-tag). Either one catching a CVE is a safety net for the other.

10. Master Command Cheat Sheet

10.1 This project's own Makefile

Command	Effect
<code>make setup</code> / <code>make bootstrap</code>	Full install: tools → Minikube → Harbor → monitoring → Jenkins → ArgoCD
<code>make clean</code>	Tear everything down (containers, Minikube, generated creds)
<code>make restart</code>	<code>clean</code> then <code>bootstrap</code> again
<code>make start</code> / <code>make stop</code>	Start/stop Minikube + Harbor + Jenkins without reinstalling
<code>make test</code> / <code>make lint</code> / <code>make security</code>	Delegate to <code>go-api</code> 's own Makefile targets
<code>make jenkins-ui</code> / <code>make harbor-ui</code> / <code>make argocd-ui</code> / <code>make grafana-ui</code> / <code>make prometheus-ui</code>	Print URL + creds, port-forward where needed
<code>make port-forward</code>	Expose dev/stage/prod app Services on 8081/8083/8084
<code>make logs-dev</code> / <code>logs-stage</code> / <code>logs-prod</code>	Tail app logs per environment

10.2 Quick reference by tool

Tool	Most-used commands
Docker	<code>docker build</code> · <code>docker run</code> · <code>docker ps</code> · <code>docker logs -f</code> · <code>docker exec -it</code> · <code>docker push/pull</code>
kubectl	<code>get</code> · <code>describe</code> · <code>logs -f</code> · <code>exec -it</code> · <code>apply -f</code> · <code>port-forward</code> · <code>rollout status/undo</code>
Helm	<code>template</code> · <code>upgrade --install</code> · <code>lint</code> · <code>list</code> · <code>rollback</code> · <code>uninstall</code>
Harbor (REST)	<code>/api/v2.0/ping</code> · <code>/api/v2.0/projects</code> · UI vulnerability tab
Jenkins	Build Now · Blue Ocean pipeline view · Manage Credentials · Console Output
ArgoCD (CLI/UI)	<code>app list</code> · <code>app get</code> · <code>app sync</code> · <code>app diff</code> · <code>app rollback</code> · UI Refresh
Prometheus	Status → Targets · Graph (PromQL) · <code>rate()</code> · <code>histogram_quantile()</code>
Grafana	Dashboards → Browse · Explore · Add panel
Trivy	<code>trivy image --severity HIGH,CRITICAL --exit-code 1 </code> · <code>trivy fs</code> .

11. Troubleshooting Playbook

Straight from this project's README — the four failure modes it explicitly documents:

Symptom	Root cause	Fix
Kubernetes can't pull images from Harbor ("insecure registry" error)	Container runtimes block plain-HTTP registries by default	Confirm Minikube was started with <code>--insecure-registry="host.minikube.internal:8082"</code> (<code>minikube profile list</code>); if missing, <code>make clean && make setup</code>
Jenkins pipeline fails to log into Harbor	<code>harbor-robot</code> credential race — Jenkins provisioned before <code>harbor-robot.env</code> existed	Re-run <code>make setup</code> (idempotent) so the credential Groovy script re-reads the current file
Jenkins can't clone/push to GitHub	<code>github-personal-ssh</code> key doesn't match the account registered on GitHub	Verify with <code>ssh -T git@github.com</code> ; override the key path with <code>PERSONAL_SSH_KEY</code> ; re-run <code>make setup</code>
ArgoCD Application shows <code>Unknown</code> / missing CRD	<code>ServiceMonitor</code> template needs Prometheus Operator CRDs, which don't exist until monitoring is installed	Run <code>scripts/install-monitoring.sh</code> , then force a re-sync in the ArgoCD UI

11.1 General debugging order-of-operations

1. `kubectl get pods -n <ns>` — is it `Running` , `CrashLoopBackOff` , `Pending` , `ImagePullBackOff` ?
2. `kubectl describe pod <pod> -n <ns>` — check the **Events** section at the bottom first.
3. `kubectl logs <pod> -n <ns>` (add `--previous` if it already restarted).
4. For scheduling issues, check node resources: `kubectl top nodes` , `kubectl describe node` .
5. For CI issues, read the failing stage's console output in Jenkins top-to-bottom — declarative pipelines stop at first failure.
6. For GitOps drift, `argocd app diff` before assuming ArgoCD "isn't working" — it may be correctly reporting a real difference.

12. Glossary

Term	Definition
CI (Continuous Integration)	Automatically building, testing, and scanning every code change
CD (Continuous Delivery/Deployment)	Automatically getting a validated build into an environment
GitOps	Using Git as the source of truth for infra/app desired state, reconciled by a pull-based controller
CRD (Custom Resource Definition)	How Kubernetes lets third parties (ArgoCD, Prometheus Operator) add their own object kinds
OCI (Open Container Initiative)	The open standard image/runtime format Docker, Harbor, and Kubernetes all speak
CVE	Common Vulnerabilities and Exposures — a public identifier for a known security flaw
SAST / SCA	Static Application Security Testing (scans your code) / Software Composition Analysis (scans your dependencies)
SBOM	Software Bill of Materials — a manifest of everything inside a built artifact
Rolling update	Replacing old Pods with new ones gradually, keeping the app available throughout
Reconciliation loop	A controller's continuous "observe → diff → act" cycle (core Kubernetes/ArgoCD pattern)
Idempotent	Safe to run more than once with the same result — a design goal of every script in this repo

— End of study guide · derived entirely from this repository's own scripts, manifests, and README —